NOMAD: In-Network Traffic Obfuscation Against Device Fingerprinting for Industrial Control Systems

Philip Akekudaga
Dept. of Electrical and Computer Engineering
University of Rhode Island
Kingston, RI, U.S.A
philip.akekudaga@uri.edu

Hui Lin
Dept. of Electrical and Computer Engineering
University of Rhode Island
Kingston, RI, U.S.A.
huilin@uri.edu

***Abstract*—TBD**
***Index Terms*—traffic obfuscation**

I. Introduction

Device fingerprinting is an essential step in cyber reconnaissance, allowing adversaries to reveal unique features of target networks and design effective attack strategies. These techniques are becoming increasingly critical in industrial control systems (ICSs) such as power grids, where adversaries often use IP-based control networks and computing devices within as entry points to initiate physical damage. Consequently, fingerprinting shifts the focus from visited websites and user biometric behavior to device models and types of control operations that are critical to ICS attacks. In the 2015 attack that disrupted Ukrainian power grids and the Stuxnet attack that disrupted Iranian nuclear power facilities, it is widely believed that adversaries stay in their systems for at least 6 months to perform cyber reconnaissance [1], [2].

To disrupt device fingerprinting, many studies present network traffic obfuscation [3], [4]. Device fingerprinting targeting general computing environments typically relies on network-level features such as packet size and/or inter-packet latency observed from communication patterns [5]. Consequently, existing traffic obfuscation focuses on (i) padding and splitting network packets, which hide or change the distribution of network packet sizes [3], and (ii) delaying network packets and adding dummy ones, which disrupt the inter-packet timing pattern [4], [6]. Because manipulating communication networks can introduce runtime overhead, recent work has begun offloading traffic obfuscation to programmable network switches, using specialized ASIC to change communication patterns at much higher line rates than CPUs [7]–[9].

Unfortunately, these methods cannot be directly applied to ICS environments for two main reasons. First, existing traffic obfuscation is performed over encrypted channels, which are necessary to mix the obfuscated and normal traffic. ICS networks, on the other hand, still rely on plain-text communications traffic due to a wide range of legacy devices, despite ICS network protocols providing security features [10]. Directly padding or adding dummy packets can easily reveal obfuscated packets, exposing the underlying traffic pattern to adversaries. Second, device fingerprinting methods on ICS devices rely on different features from the ones in general computing environments. Instead of using network layer knowledge, they attempt to use behavior on physical devices to reveal device model or types of operations. For example, work in [11] uses cross-layer response time (CLRT), determined by the latency between the TCP acknowledge packets (ACK) and the actual response from the same device to estimate execution time there. Since each vendor may choose different materials or algorithms to implement the same control functionality, CLRT becomes an effective feature for ICS device fingerprinting.

In this paper, we present NOMAD, an in-network traffic obfuscation mechanism against ICS device fingerprinting, even with plain-text communication. Consequently, adversaries cannot conduct effective reconnaissance using side-channel knowledge to reveal target systems' vulnerabilities (e.g., outdated firmware or vulnerable device models) from network communications. Our solution draws on a key insight: to carry *control units* (e.g., measurements and commands), ICS network protocols map them to numerical indices, with syntax varying across control operations. Instead of applying the same obfuscation strategy to each network packet (as in many existing solutions), we use programmable network switches to extract operation-related knowledge from network traffic and apply different strategies by splitting, padding, and scheduling control units while maintaining protocol compliance. NOMAD realizes multiple advancements with small runtime overhead: (i) provide flexible obfuscation (e.g., normalize TCP ACK and application response in two different patterns); (ii) preserve communication with legitimate end devices; and (iii) require no reliance on de-obfuscation hardware and encryption over ICS traffic.

Unlike previous solutions which applies the same traffic obfuscation strategies to all network packets, that normalizes an outstation's fingerprintable features using the data plane of the Tofino programmable switch at the outstation edge, without changing the endpoints or modifying the bytes of the DNP3 traffic. This defense holds per-transaction responses and releases them at configurable offsets, so the master-visible timing then becomes a policy value. We implement this as a P4 program on a single switch and evaluate it on a cyberphysical testbed and make the following contributions:

- We propose an in-network turning framework that obfus-

cates timing features and shapes how an on-path passive observer fingerprints outstation devices. This, to the best of our knowledge, is the first in-network defense for ICS device-fingerprinting features.

- A case study of the timing frameworks introduced by Formby et al [11], we present a multi-configurable mode of operation that holds TCP acknowledgments and DNP3 Responses and releases at predetermined offsets from the request, such that the master visible response time is deterministic
- Normalizes the control command and operation transaction time. As a second case study covering the physical fingerprinting feature of Formby et al. We build a hold mechanism to hold and determine the observed time on the master-facing observed traffic and show the passive observer records a time that is obfuscated.
- We evaluate this mechanism on a single physical Tofino and outstation device.

II. Background

A. ICS Network Communications

Like many ICSs, power grids comprise cyber and physical infrastructures. *Physically*, substations (buses) are linked by electrical lines into a transmission or distribution network. The *cyber* infrastructure connects field devices, e.g., IEDs (intelligent electronic devices), PLCs (programmable logic controllers), and legacy sensors and breakers, to a control center over an IP-based *control network*. The control center uses supervisory control and data acquisition (or a similar setup) to periodically collect measurements (voltage, current, power consumption) and issue commands to maintain stable, safe physical processes.

ICS network protocols, e.g., DNP3 and Modbus, are designed mainly to carry control units, e.g., measurements or control commands, to accomplish these functions. The syntax of control units varies across operational types. For example, the DNP3 protocol specifies primitive types, e.g., floating points, to represent measurements and composite types, e.g., control relay output block (CROB), to represent commands applied to circuit breakers. The protocol further uses numerical indices to organize various control units in a single network packet. The protocol-conforming devices, e.g., IEDs and PLCs, store a local configuration file, to look up the meaning of a control unit based on its indices.

B. ICS Device Fingerprinting

Because physical devices may use different algorithms or mechanical components to implement the same control operations, adversaries can use side-channel knowledge from control networks, e.g., packet size or timing, to pinpoint specific device types and control operations. This information can reveal vulnerable device models or firmware versions, enabling effective attacks. For example, studies in [11] use two proprietary features for device fingerprinting, as shown in Fig. 1. Cross-layer response time (CLRT), calculated by the latency between a TCP ACK packet and the corresponding application layer response, estimates the time used by specific IEDs to retrieve measurements. Physical Response Time (PRT), calculated as the latency between unsolicited responses (indicating the start of a control operation) and the corresponding application-layer response, estimates the time specific IEDs take to execute control commands. These device fingerprinting presents different features compared to website or browser fingerprinting in general computing environments. While the latter treat every packet equally (as they are encrypted), device fingerprinting refers to timing between different types of packets to reveal device-specific features.

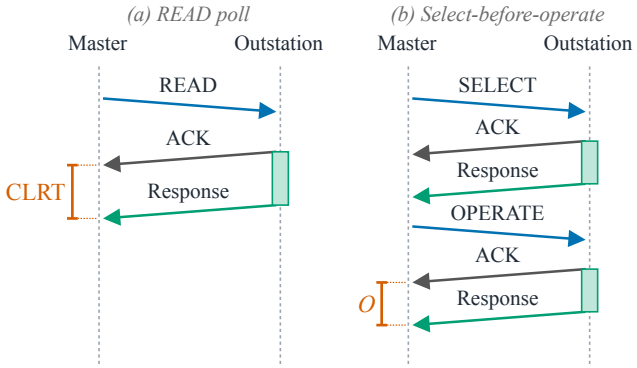


Fig. 1. The two DNP3 transactions on the master-facing link. (a) A READ poll and its CLRT. (b) A select-before-operate control and its master-visible interval O.

[Archive] We first present the DNP3 transactions captured in the scope of this paper, and then explain why the timing of these transactions serves as a fingerprint for the device. We then go into who a we able to change that.

[Archive] DNP3 carries traffic over TCP between the master and outstation [10]. In figure 1, we show the transactions where the master sends a poll READ request with a function code 1 and the outstation replies with an acknowledgment and the measurements as a response with a function code 0X81. A control command uses a select-before-operate process where the master first selects the function code (3 in the out case) with the name of output indices, and the outstation confirms it. With the acknowledgment, the master proceeds with an OPERATE function, which the operator executes and confirms with a solicited response.

[Archive] For a DNP3 protocol that is carried over TCP, it uses the transport layer if the outstation acknowledges each request it receives. This acknowledgment travels in segments before the application layer answers, even though the TCP allows it to ride on the response [12]. The exchange therefore appears as three packets per transaction: a request, a transport layer acknowledgment that carries not the DNP3 payload, and then a DNP3 response.

[Archive] Definition 1. Cross-Layer Response Time (CLRT): This is the measured time interval between the arrival of the transport acknowledgment and the arrival of the DNP3 response of the same transaction.

[Archive] Definition 2. Physical Operation Time: For

a select-before-operate command, the measure time interval between the OPERATE command and when the outstation reports the physical change execution at the device.

C. *[Archive] Execution Time as a Fingerprint*

[Archive] An outstation answers a request in two transactions that comes from two different layers. Its TCP acknowledges the request as it arrives, while the application layer responds after the device is done with the request. Because both packets travel on the same path and in the same direction, there is no propagation delay, and consequently, the measured CLRT will be the execution time of the outstation device.

[Archive] The execution times differ for devices and models, and since different vendors have their own implementations for the same functions but with different hardware and algorithms, Formby et al. classifies DNP3 device types and models by their distribution of CLRT [11] and Lin et al. measured the execution times of devices and separated them by combining their averages and variations. For instance, the average CLRT of a SEL 751A is 2.929 ms for a control command and 2.121 ms for a poll, while its median acknowledgment latencies for READ, SELECT, and OPERATE lie within 0.003 ms of one another. Consequently, the CLRT carries how this device performs an operation, which is the profile a fingerprinting adversary builds.

D. Network Switch with Programmable Network Dataplane

Network switches with programmable network data plane enabled by P4 draw significant attention to host in-network security analytics. P4 stands for Programming Protocol-independent Packet Processors and is the de facto data-plane programming language enable customized packet-processing functions [13]. As shown in Fig. **??**, Portable Switch Architecture (PSA) provides reference infrastructure for a programmable network data plane. It can feature up to four ingress and four egress pipelines [**?**], each composed of a parser, a deparser, and a sequence of match/action units (MAUs). MAUs include various storage units (e.g., SRAM, TCAM) to store network knowledge and arithmetic logic units (ALUs) to enable the following features directly in network devices: customized packet parsing across all network layers, line-rate analysis, and integrated monitoring and traffic engineering. However, realizing these benefits requires meeting with strict hardware limits, e.g., roughly 120 MB of SRAM, 10 MB of TCAM, and ALUs that do not support floating-point operations, multiplication, or division.

[Archive] A programmable switch offers a match-action pipeline coded in P4 [13] and parses headers, keeps states in registers, and has the ability to keep packets in buffers and queues. As such, the pipeline can delay a packet and release it later without a host in the loop; a switch place between the master and the outstation can hold the acknowledgment and the response of a transaction and release them on a schedule the operator sets. With that, the switch can change the timing an adversary observes without changing either endpoint or any byte of the payload.

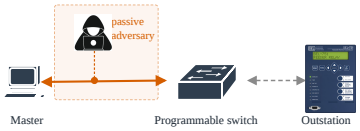


Fig. 2. Observation model. The adversary monitors the shaded master-facing link. The dashed paths are not visible to it.

III. NOMAD Design Overview

In this section, we first present the threat model used by current ICS device fingerprinting. The assumptions about adversaries lead to NOMAD's design principles and major building components.

A. Threat Model

Assumptions on Adversary's Capability. We assume the adversary has penetrated the control network already and can passively monitor traffic from the master-facing link as shown in Figure 2. The passive observer is able to read and record packet headers, their sizes, and arrival time, and because DNP3 does not have any transport security by default [14], this gives the observer the ability to read the function codes of each request, and using that, they are able to label each exchange by what it does, e.g., READ poll or SELECT and OPERATE. We also assume that the adversary knows the existence of the framework and the intervals it obfuscates, but they cannot read the internal mechanism and state of the switch or be able to monitor the link between the switch and the outstation. This is practical, as the switch usually sits at the edge of the outstation and is usually connected with a single cable between devices in the same substation cabinet.

Adversary's Goal. The aim of the adversary is to fingerprint the outstation using the timing from the exchanges and then infer the make and model of devices and the operation each of them performs. With this knowledge, they can design specific and targeted attacks like false data injection that must match the devices they compromise [15]. We consider an adversary that builds its classifier from captured traffic before the framework is deployed and applies it afterwards.

B. Design Goals

Before describing NOMAD's design goal, we use a sample network trace collected from IEDs from two different vendors, i.e., the Schweitzer Engineering Laboratories (SEL) 751A [16] and the Schneider Electric (SE) ION7550 power meter [17], to demonstrate unique characteristics in ICS device fingerprinting. First, despite establishing security extensions in common ICS network protocols, plain-text communication is still prevalent in power grids and other ICSs. We have developed a web crawler that examined public product manuals, datasheets, and pages from five IED vendors, namely, SEL, SE, Rockwell Automation, GE Vernova, and Siemens. With a stratified sample of 99 catalogued entries, 32 were identified as having DNP3 support, and these documents established encryption for six entries, and one had DNP3 authentication.

However, it is also established that OT asset owners and operators do not use these security measures even if they exist, with numerous reasons hindering their adoption [18]. The crawler methodology and detailed results are presented in Appendix **??**.

Second, different IEDs can configure network protocols differently, and some even include undocumented variants. These configurations, largely benefiting from plain-text communication, can enable ad hoc features for ICS device fingerprinting. In Figure **??**, we present the data acquisition based on the DNP3 protocol on IEDs from two different vendors, i.e., the Schweitzer Engineering Laboratories (SEL) 751A [16] and the Schneider Electric (SE) ION7550 power meter [17]. Based on the network trace, we find that SEL 751A disables TCP's "delayed ACK" mechanism, making CLRT a viable fingerprinting feature. However, SE ION7550 enables this mechanism, so we can no longer use CLRT to estimate how long the device spent retrieving measurements. and configure the DNP3 protocol with different parameters. We believe the proprietary network protocol implementation and configuration can form unique device fingerprints, which also present an excellent opportunity to disrupt adversaries' reconnaissance if a different device fingerprint can be manipulated.

To counter unique ICS device fingerprinting, we design NOMAD driven by the following goals.

- **Obfuscation over Plain-text.** Despite ICS network protocols are gradually enhanced with security components, plain-text communication still prevails in ICS due to existing legacy devices and demanding communication requirements. It is essential to apply various obfuscation strategies, including packet splitting and padding, over plain-text communication without revealing the fact.
- **Flexible Obfuscation.** Features from different types of network packets can combine to form unique fingerprints for IEDs. Applying different obfuscation strategies based on packet types and extracted features is important to accommodate ad-hoc fingerprinting methods.
- **Low Runtime Overhead.** Maintaining constant latency while applying various obfuscation strategies on thousands of IEDs in a power grid is essential to disrupt diverse device fingerprinting.
- **No Reliance on De-obfuscation.** If we follow existing obfuscation solutions, a de-obfuscation unit processes network traffic from obfuscated traffic from all IED, presenting security and overhead bottleneck. It is critical to make obfuscation directly protocol-conformance, putting no reliance on de-obfuscation.

[Archive] The Framework's Objective.. Our aim is to disrupt the timing-based fingerprinting of the outstations without changing the outstations, the master, or the DNP3 protocol in use. We thereby work within the following research objectives:

- **R01:** For both read and command transactions, we aim at making the CLRT the adversary calculate a value that the operator sets and not dependent on the execution time of the outstation.
- **R02:** For both fixed and adaptive adversaries, it is our goal to remove the execution time of the outstation and what they learn such that either the fixed or adaptive adversary's classification is thrown off and whatever they learn reflects that of the switch instead of the device.
- **R03:** we aim to have the framework work with negligible overhead to the operations of the control network. We do not add packets or bytes, and we aim at keeping added delays within TCP and DNP3 timers to prevent retransmission of requests.

Trusted Components. To make these objectives possible, we only add a switch and a control plane to the trusted computing base. The control plane installs the policy to the switch for enforcement. The master or the outstation is not needed, and neither are they modified for the framework to function.

C. Components and Workflows

As shown in Fig. **??**, NOMAD includes two components: (i) *Obfuscation infrastructure* that uses P4-enabled network data planes to apply flexible obfuscation strategies and (ii) *obfuscation objectives/constraints* determining security properties in various scenarios while observing ICS communication requirements. When a network packet traverses a gateway network device, we use a customized parser to extract network attributes and control-related knowledge, then categorize packets and organize the carried control units accordingly in the switch's local storage. Using the control-unit syntax, including sizes and indices, we can split, pad, and schedule control units according to different operations while maintaining protocol compliance. While obfuscation infrastructures provide an in-network hardware foundation, we use switches' control plane to realize various obfuscation objectives under constraints for diverse security applications, e.g., neutralize, mimicry communication patterns, and/or maximize differences between obfuscated and original patterns.

[Archive]

IV. Design

In this section, we describe the timing model and the packet flow through the mechanism. We will then define the design variables and parameters along with the conditions under which the framework follows the schedule. Lastly, we explain the queues and the P4 pipeline used for this schedule.

A. Timing Model and Packet Flow

In order to change the timing the adversary observes, we place a programmable switch between the master and the outstation. When a READ poll or a SELECT passes through the switch, it forwards the request and holds the return acknowledgment and response, which carries the timing that will be seen by the adversary. For OPERATE, it also holds the command before forwarding it to the relay. Consequently, both cases use the request arrival time to schedule the returning packets such that the release deadlines do not follow the outstation acknowledgment latency.

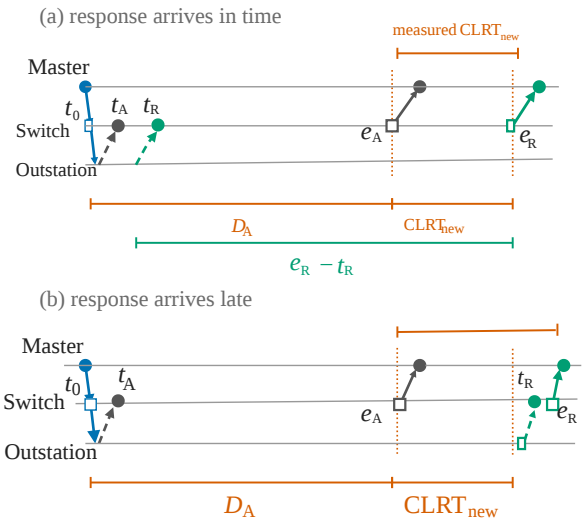


Fig. 3. Release timeline of one READ transaction. All five instants are at the switch: the request arrives at t_0, the acknowledgment and response arrive at t_A and t_R, and the switch releases them at e_A and e_R. The schedule is anchored at t_0. (a) The response arrives before its deadline. (b) It arrives late and leaves when its queue next serves it. The drawing illustrates the ideal schedule; actual release also includes queue service.

Timing Model. In Figure 3, we show the timing of a read transaction. We denote the request arrival time at the switch by t_0, the acknowledge and response arrival times as t_A and t_R , and the time they depart to the master as e_A and e_R. With this, the original acknowledgment-to-response is

$$\text{CLRT}_{\text{original}} = t_R - t_A. \tag{1}$$

holding the two packets changes that interval to

$$e_R - e_A = \text{CLRT}_{\text{original}} + (e_R - t_R) - (e_A - t_A), \tag{2}$$

where $e_A - t_A$ and $e_R - t_R$ are the acknowledgment and response hold times, respectively. Because the adversary sits on the master facing, they will measure a CLRT where the difference in path and capture delay between the two packets also contributes to the interval. We distinguish that measurement from the switch release interval when interpreting the results.

Replacing the interval. A constant hold on each packet will only shift the original CLRT by the difference between the holds, which will reserve the variation that a timing classifier can use. In order to absorb that variation, we choose to release the deadlines from the request such that a packet arriving early has to be longer and a packet arriving closer to the deadline waits less. This design choice will require both packets to arrive before their respective deadlines.

Holding with Queues. In order to keep packets in hold, we place them in a queue below another queue of higher priority. The higher queue contains internal *blocker packets*,which the scheduler serves instead of the held acknowledgment and response packets so far as the higher priority queue remains non-empty. We circulate the blocker packets until their deadline, after which they stop recirculating to fill the queue. This drains the remaining blockers and then allows the scheduler to serve the held packets. This way, the queues provide the timing and the wait while the P4 programs decide which packets enter them and when blockers drain.

B. Design Parameters and Constraints

Release Deadlines. We configure two parameters, D_A as the acknowledgment deadline that is measured from the request, and CLRT_{new}, the desired gap that an adversary will measure as the interval between an acknowledgment and the response. When a request arrives, we set

$$\hat{e}_A = t_0 + D_A, \qquad \hat{e}_R = t_0 + D_A + \text{CLRT}_{\text{new}}. \tag{3}$$

the hats to denote scheduled parameters, not measured departures. We call their combined offset

$$D = D_A + \text{CLRT}_{\text{new}} \tag{4}$$

the response *release budget*

Packet Availability. The switch will not be able to release a packet it has not received; the ideal schedule will require that

$$t_A \le t_0 + D_A, \qquad t_R \le t_0 + D. \tag{5}$$

under these conditions, the ideal holds are

$$\begin{aligned} \hat{e}_A - t_A &= D_A - (t_A - t_0), \\ \hat{e}_R - t_R &= D - (t_R - t_0). \end{aligned} \tag{6}$$

With this, the hold is able to absorb the changes in the arrival time without moving the scheduled departures. If either packet arrives later than scheduled, its release then depends on when it arrives and when the queue can serve it. To maintain both deadlines, we must first admit the transaction and be able to sustain each packet held with enough blockers.

Actual Release. For any admitted transaction that arrives on time and whose hold last until their deadline, we write $e_A = \hat{e}_A + \varepsilon_A$ and $e_R = \hat{e}_R + \varepsilon_R$. There are release tails that include the time for the remaining blockers in the hold queues to drain and for priority to shift to the held response packets to be released by the traffic manager. Therefore,

$$\begin{aligned} e_A - t_0 &= D_A + \varepsilon_A, \\ e_R - e_A &= \text{CLRT}_{\text{new}} + \varepsilon_R - \varepsilon_A. \end{aligned} \tag{7}$$

consequently, the actual release gap differs from the configured time value by the difference between the two release tails that the operator sets.

1) Control Transactions: To separate OPERATE from its arrival, we configure a set of queues to hold the commands at J. When an OPERATE arrives at $T_0$the switch

V. Implementation

To evaluate this defense, we implement it on UfiSpace running Intel Tofino-1 ASIC, in between a Dell PowerEdge R440 as a master and a SEL-751A sitting in the outstation as our testbed.

Target and program. We design a single $P4_16$ program for the Tofino Native Architecture on the Tofino switch and

compiled it with Barefoot SDE 9.13. The program compiles in twelve ingress match action stages and six egress stages with 112 tables. We

[Archive]

VI. Related Work

Device fingerprinting. Devices can be identified by what they emit and communicate even without access to the payload. Kohno et al. fingerprinted a remote host from the skew of its clock as seen in its TCP timestamps [19], and GTID fingerprints a device and its type using the timing signature of its packets [20]. These earlier works identified the threats we address, but our objective is to remove the timing features they rely on from the master-facing observation where a passive adversary sits.

ICS and power-systems fingerprinting. Formby et al. were first to introduce fingerprinting to control systems and define what a cross-layer response time and physical operation time measured on a DNP3 outstation device are [11], and Gu et al. added device physics as a feature [21]. Jeon et al. fingerprint SCADA devices passively without deep packet inspection [22]. These methods that this technique presents help in the execution of false-data injection and process control attacks [15], [23], [19], [20], [21], [22], [23]. Many defenses have been presented and have so far worked at the content and connectivity layer. DefRec virtualizes physical functions and presents deceptive measurements [24], while RAINCOAT randomizes data acquisition and and injects decoy measurements [25] and a testbed that evaluates such anti-reconnaissance approaches on grid infrastructure [26]. In comparison to these defenses, our mechanism works below the semantics and reshapes when the outstation answers and what the outstation's answer says.

General traffic obfuscation. The mechanism of traffic morphing has been used to transform one class of packet size distribution into another [3], while Dyer et al proved that per-packet padding and fragmentation still leave features that can be exploited [4].Tamaraw presented fixes for packet size and rates at the cost of bandwidth [6] and WTF-PAD uses padding to fix interpacket gaps adaptively [27]. However, a good fingerprinting attack defeats several of these techniques [5] and questions their accuracy at an internet scale [28]. Size and timing channels have been used in identifying smart-home devices even with encryption, where shaping is a defense measure [29],whiles Pacer and NetShaper shape traffic to a particular schedule that bounds side-channel leakage in the cloud [30], [31]. Ditto, more recent implementation defines a size pattern whiles using a queue to regulate the rate that is seen and uses two switches on both sides [7] All these defenses assume an encrypted payload and cooperating end host or hypervisor that can add covert traffic. In contrast, this work targets plaintext protocols commonly found in legacy ICS whose feature is one interval inside a request-response exchange without adding bytes and must run in a setting where the outstation is fixed and cannot be changed.

Programmable data-plane traffic transformation. P4 defines a programmable parser and a match-action model that we make use of [13]. PIFO defines a programmable scheduling at line rate [32] and SP-PIFO uses this to estimates priority queues on today's switches [33]

References

[1] R. M. Lee, M. J. Assante, and T. Conway, "Analysis of the cyber attack on the Ukrainian power grid," Electricity Information Sharing and Analysis Center (E-ISAC) and SANS Institute, Defense Use Case, Mar. 2016, tLP: White, 18 March 2016.

[2] R. Langner, "Stuxnet: Dissecting a cyberwarfare weapon," *IEEE Security & Privacy*, vol. 9, no. 3, pp. 49–51, 2011.

[3] C. V. Wright, S. E. Coull, and F. Monrose, "Traffic morphing: An efficient defense against statistical traffic analysis," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2009.

[4] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, "Peek-a-boo, I still see you: Why efficient traffic analysis countermeasures fail," in *Proc. IEEE Symp. on Security and Privacy (S&P)*, 2012, pp. 332–346.

[5] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2018, pp. 1928–1943.

[6] X. Cai, R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg, "A systematic approach to developing and evaluating website fingerprinting defenses," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2014, pp. 227–238.

[7] R. Meier, V. Lenders, and L. Vanbever, "Ditto: WAN traffic obfuscation at line rate," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2022.

[8] Z. Wang *et al.*, "Minos: A lightweight and dynamic defense against traffic analysis in programmable data planes," in *Proc. USENIX Annual Technical Conference (ATC)*, 2025.

[9] G. Xie *et al.*, "Securitas: Defending against traffic analysis attacks with flexible in-network obfuscation," in *Proc. 23rd USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, 2026.

[10] IEEE, "IEEE standard for electric power systems communications—distributed network protocol (DNP3)," IEEE Std 1815-2012, 2012, iEEE Std 1815-2012.

[11] D. Formby, P. Srinivasan, A. M. Leonard, J. D. Rogers, and R. A. Beyah, "Who's in control of your control system? Device fingerprinting for cyber-physical systems," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2016.

[12] W. Eddy, "Transmission control protocol (TCP)," RFC 9293, Internet Engineering Task Force, 2022.

[13] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker, "P4: Programming protocol-independent packet processors," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, 2014.

[14] S. East, J. Butts, M. Papa, and S. Shenoi, "A taxonomy of attacks on the DNP3 protocol," in *Critical Infrastructure Protection III (IFIP AICT, Vol. 311)*. Springer, 2009, pp. 67–81.

[15] Y. Liu, P. Ning, and M. K. Reiter, "False data injection attacks against state estimation in electric power grids," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2009, pp. 21–32.

[16] Schweitzer Engineering Laboratories, *SEL-751A Feeder Protection Relay Instruction Manual*, 2013, pM751A-01-NB, 20130329 printing; settings table, `STIMEO1` select/operate time-out.

[17] Schneider Electric, "PowerLogic ION7550/ION7650 Series," accessed: Oct. 15, 2026. [Online]. Available: https://www.se.com/us/en/product-range/1460-powerlogic-ion7550-ion7650-series/

[18] Cybersecurity and Infrastructure Security Agency, "Barriers to Secure OT Communication: Why Johnny Can't Authenticate," Cybersecurity and Infrastructure Security Agency, Guidance, February 2026, published February 10, 2026. [Online]. Available: https://www.cisa.gov/sites/default/files/2026-02/Barriers-to-Secure-Communication-Why-OT-Johnny-Cant-Authenticate_508_2.pdf

[19] T. Kohno, A. Broido, and K. C. Claffy, "Remote physical device fingerprinting," *IEEE Trans. on Dependable and Secure Computing*, vol. 2, no. 2, pp. 93–108, 2005.

[20] S. V. Radhakrishnan, A. S. Uluagac, and R. Beyah, "GTID: A technique for physical device and device type fingerprinting," *IEEE Transactions on Dependable and Secure Computing*, vol. 12, no. 5, pp. 519–532, 2015.

[21] Q. Gu, D. Formby, S. Ji, H. Cam, and R. Beyah, "Fingerprinting for cyber-physical system security: Device physics matters too," *IEEE Security & Privacy*, vol. 16, no. 5, pp. 49–59, 2018.

[22] S. Jeon, J.-H. Yun, S. Choi, and W.-N. Kim, "Passive Fingerprinting of SCADA in Critical Infrastructure Network without Deep Packet Inspection," Aug. 2016.

[23] A. A. Cárdenas, S. Amin, Z.-S. Lin, Y.-L. Huang, C.-Y. Huang, and S. Sastry, "Attacks against process control systems: Risk assessment, detection, and response," in *Proc. 6th ACM Symp. on Information, Computer and Communications Security (ASIACCS)*, 2011, pp. 355–366.

[24] H. Lin, J. Zhuang, Y.-C. Hu, and H. Zhou, "DefRec: Establishing physical function virtualization to disrupt reconnaissance of power grids' cyber-physical infrastructures," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2020.

[25] H. Lin, Z. T. Kalbarczyk, and R. K. Iyer, "RAINCOAT: Randomization of network communication in power grid cyber infrastructure to mislead attackers," *IEEE Transactions on Smart Grid*, vol. 10, no. 5, pp. 4893–4906, 2019.

[26] H. Lin, H. Alemzadeh, D. Chen, Z. Kalbarczyk, and R. K. Iyer, "Safety-critical cyber-physical attacks: Analysis, detection, and mitigation," in *Proc. Symp. and Bootcamp on the Science of Security (HotSoS)*, 2016, pp. 82–89.

[27] M. Juarez, M. Imani, M. Perry, C. Diaz, and M. Wright, "Toward an efficient website fingerprinting defense," in *Proc. European Symp. on Research in Computer Security (ESORICS)*, 2016, pp. 27–46.

[28] A. Panchenko, F. Lanze, J. Pennekamp, T. Engel, A. Zinnen, M. Henze, and K. Wehrle, "Website fingerprinting at internet scale," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2016.

[29] N. Apthorpe, D. Y. Huang, D. Reisman, A. Narayanan, and N. Feamster, "Keeping the smart home private with smart(er) IoT traffic shaping," *Proceedings on Privacy Enhancing Technologies (PoPETs)*, vol. 2019, no. 3, pp. 128–148, 2019.

[30] A. Mehta, M. Alzayat, R. De Viti, B. B. Brandenburg, P. Druschel, and D. Garg, "Pacer: Comprehensive network side-channel mitigation in the cloud," in *Proc. 31st USENIX Security Symposium*, 2022.

[31] A. Sabzi, R. Vora, S. Goswami, M. Seltzer, M. Lécuyer, and A. Mehta, "NetShaper: A differentially private network side-channel mitigation system," in *Proc. 33rd USENIX Security Symposium*, 2024.

[32] A. Sivaraman, S. Subramanian, M. Alizadeh, S. Chole, S.-T. Chuang, A. Agrawal, H. Balakrishnan, T. Edsall, S. Katti, and N. McKeown, "Programmable packet scheduling at line rate," in *Proc. ACM SIGCOMM Conference*, 2016, pp. 44–57.

[33] A. G. Alcoz, A. Dietmüller, and L. Vanbever, "SP-PIFO: Approximating push-in first-out behaviors using strict-priority queues," in *Proc. 17th USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, 2020, pp. 59–76.